Introduction to Artificial Intelligence
Final Project Report

Yuval

March 12, 2026

GitHub: `https://github.com/Yuvalinnnn/intro-to-ai-project`

Contents

1 Regression Task: Fuel Efficiency Prediction **3**
1.1 Dataset and Preprocessing . . . 3
1.2 Exploratory Data Analysis . . . 3
1.3 Linear Regression Baseline . . . 4
1.4 Polynomial Regression . . . 5
1.5 KNN Regression . . . 6
1.6 Optimization Behavior . . . 6
1.7 Model Comparison and Final Evaluation . . . 7

2 Classification Task: Classical Classification on CIFAR-10 **7**
2.1 Dataset and Preprocessing . . . 7
2.2 Logistic Regression . . . 8
2.3 Support Vector Machine (SVM) . . . 9
2.4 KNN Classification . . . 10
2.5 Model Comparison and Final Evaluation . . . 11

3 Neural Network with PyTorch **12**
3.1 Overview . . . 12
3.2 Barebones PyTorch (Part II) . . . 12
3.3 Module API (Part III) . . . 12
3.4 Sequential API (Part IV) . . . 12
3.5 Open-Ended Challenge – BetterNet (Part V) . . . 12
3.6 Results . . . 13

4 Bonus: Neural Network Regression with PyTorch **14**
4.1 Overview . . . 14
4.2 Architecture and Hyperparameter Search . . . 14
4.3 Final Model . . . 14
4.4 Comparison with Part 1 . . . 15

Introduction

This report presents the implementation and analysis of three supervised learning tasks as part of the Introduction to Artificial Intelligence final project.

Part 1 focuses on regression using the Auto MPG dataset, where we predict fuel efficiency using Linear Regression, Polynomial Regression, and KNN Regression, and analyze optimization behavior.

Part 2 focuses on classical classification on the CIFAR-10 dataset using Logistic Regression, SVM, and KNN, evaluating their performance on high-dimensional image data.

Part 3 implements neural network classifiers using PyTorch at three levels of abstraction, culminating in a custom architecture (BetterNet) that achieves 53.7% test accuracy.

Bonus adapts the PyTorch framework to solve the regression problem from Part 1 using a neural network, achieving a test RMSE of 2.340.

All code is available at: `https://github.com/Yuvalinnnn/intro-to-ai-project`

1 Regression Task: Fuel Efficiency Prediction

1.1 Dataset and Preprocessing

We used the Auto MPG dataset, containing 398 samples and 7 features: `cylinders`, `displacement`, `horsepower`, `weight`, `acceleration`, `model_year`, and `origin`. The target variable is `mpg` (miles per gallon).

The `horsepower` feature contained 6 missing values, which were filled with the median value. All features were standardized using `StandardScaler`. The dataset was split into train (70%), validation (15%), and test (15%) sets.

1.2 Exploratory Data Analysis

The target variable `mpg` ranges from 9 to 46.6 with a mean of 23.5. The distribution is right-skewed, indicating that most cars have relatively low fuel efficiency.

The correlation heatmap revealed strong negative correlations between `mpg` and `weight` ($r = -0.83$), `displacement` ($r = -0.80$), `cylinders` ($r = -0.78$), and `horsepower` ($r = -0.78$). A positive correlation was found with `model_year` ($r = 0.58$), suggesting that newer cars tend to be more fuel-efficient.

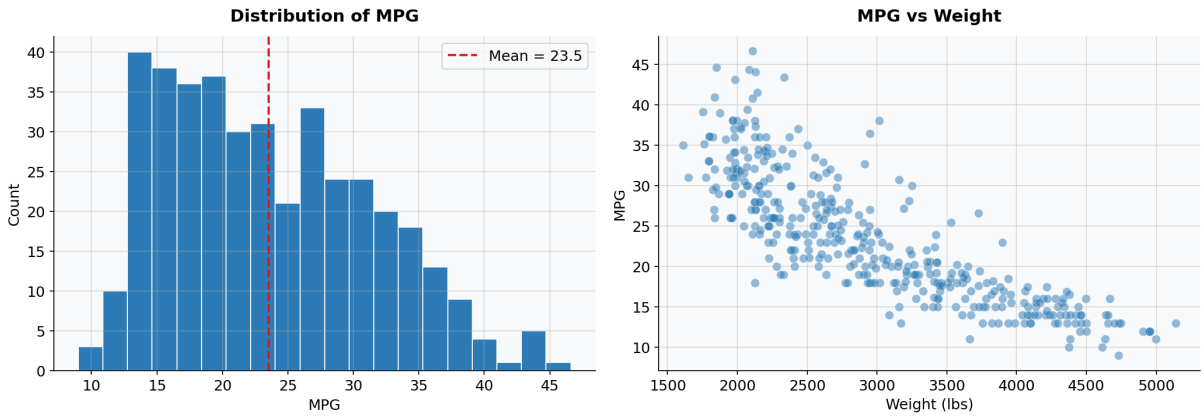

Figure 1: Distribution of MPG (left) and MPG vs Weight (right).

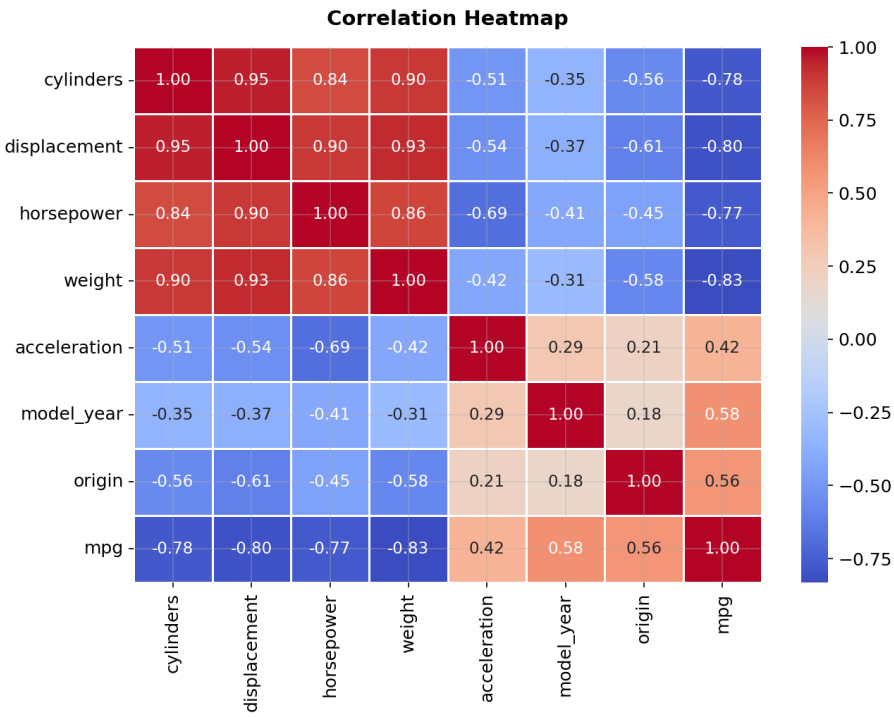

Figure 2: Correlation Heatmap of all features.

1.3 Linear Regression Baseline

A linear regression model was trained as a baseline. Table 1 reports its performance on the validation set.

Metric	Value
MSE	8.929
RMSE	2.988
MAE	2.360
R^2	0.818

Table 1: Linear Regression – Validation Metrics

The model explains 81.8% of the variance in the data. The linearity assumption appears reasonable given the strong linear correlations observed in the EDA. However, the model slightly underestimates high MPG values, suggesting some non-linearity in the data.

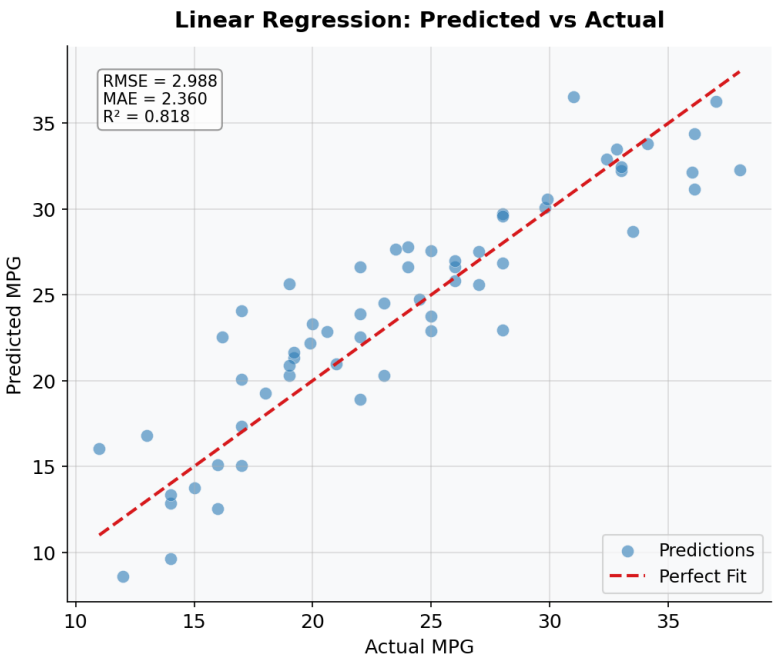

Figure 3: Linear Regression: Predicted vs Actual MPG.

1.4 Polynomial Regression

To capture non-linear relationships, polynomial regression models of degrees 1–5 were evaluated. As shown in Figure 4, degree 2 achieves the best validation RMSE of 2.850. Degrees 4 and 5 show severe overfitting, with training RMSE of 0 but validation RMSE exceeding 80.

The selected model (degree 2) improves upon the linear baseline while avoiding overfitting, demonstrating a better bias–variance tradeoff.

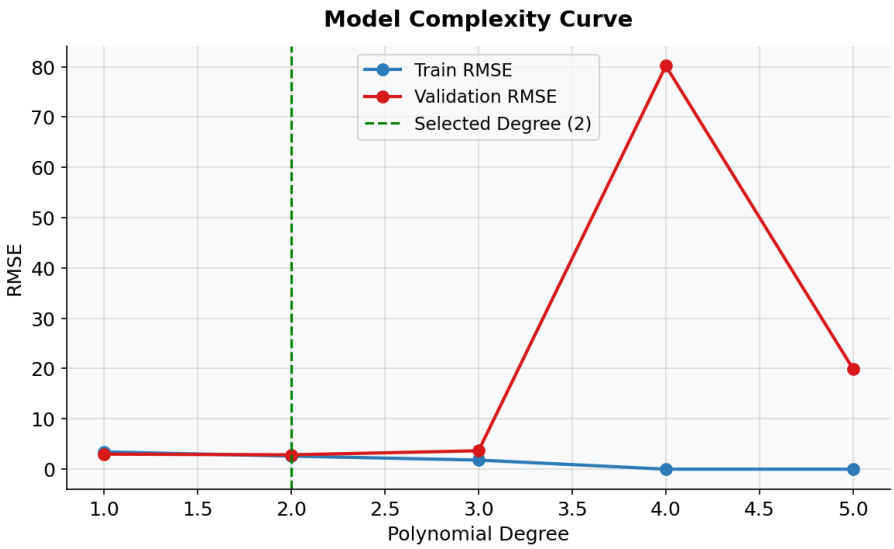

Figure 4: Model Complexity Curve: Train vs Validation RMSE by polynomial degree.

1.5 KNN Regression

A KNN regression model was evaluated for $k = 1, \ldots, 20$. The best validation performance was achieved at $k = 4$ (RMSE $= 2.578$, $R^2 = 0.865$). Small values of k lead to overfitting (at $k = 1$, train RMSE $= 0$), while large values lead to underfitting.

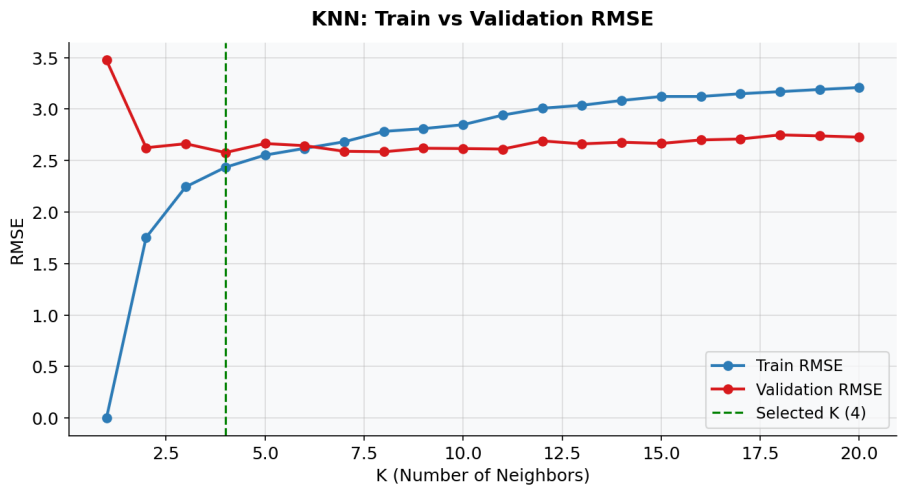

Figure 5: KNN: Train vs Validation RMSE for different values of K.

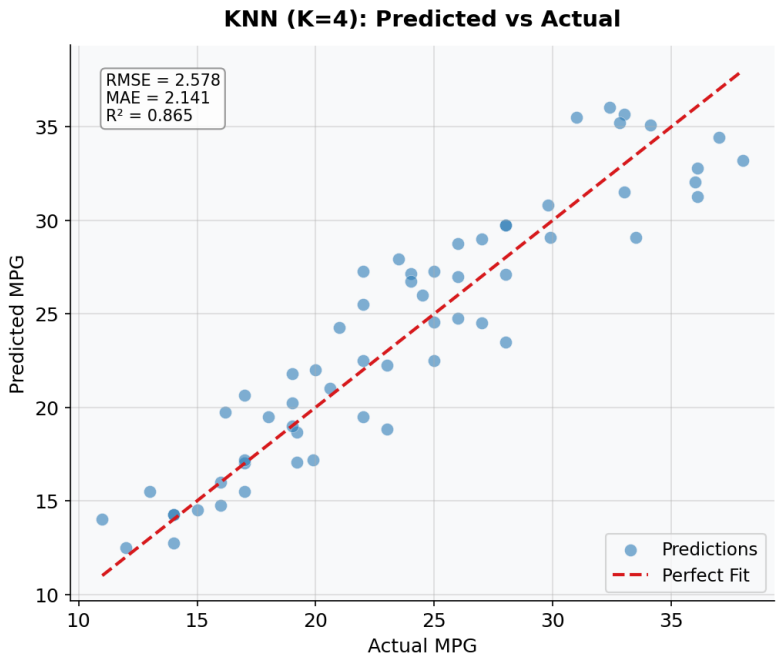

Figure 6: KNN (K=4): Predicted vs Actual MPG.

1.6 Optimization Behavior

Batch Gradient Descent converges immediately to a stable solution (RMSE ≈ 3.0), while SGD starts higher and converges more slowly, stabilizing at a slightly higher error ($\approx$ 3.05). Batch GD is more stable due to using the full dataset at each step, while SGD is noisier but more scalable to large datasets.

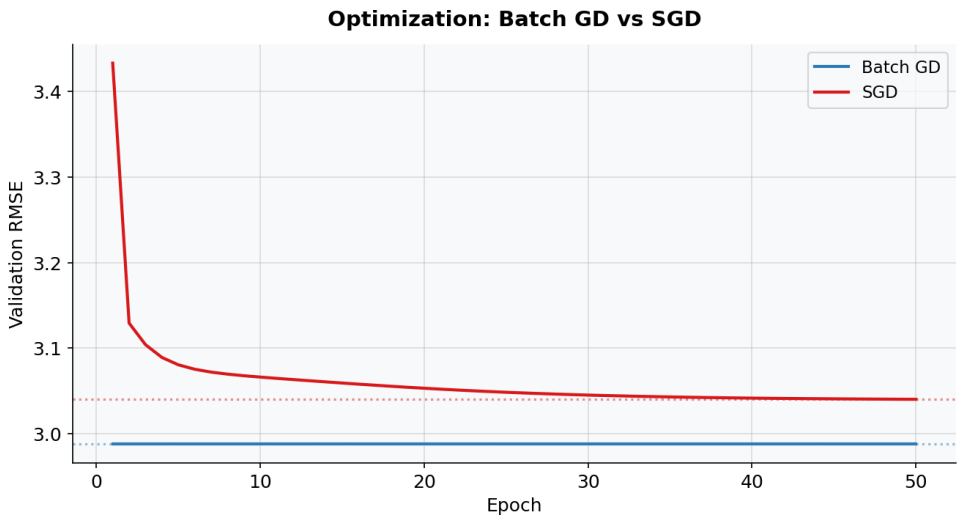

Figure 7: Optimization: Batch GD vs SGD over 50 epochs.

1.7 Model Comparison and Final Evaluation

Table 2 summarizes all models on the validation set.

Model	**RMSE**	**R^2**
Linear Regression	2.988	0.818
Polynomial (degree=2)	2.850	0.834
KNN ($k = 4$)	2.578	0.865

Table 2: Model Comparison – Validation Set

KNN ($k = 4$) was selected as the final model based on its superior validation performance. Evaluated on the held-out test set, it achieved RMSE = 2.400 and $R^2 = 0.912$, confirming strong generalization.

2 Classification Task: Classical Classification on CIFAR-10

2.1 Dataset and Preprocessing

The CIFAR-10 dataset consists of 60,000 color images of size 32×32 pixels, belonging to 10 classes: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck. Each image is represented as a vector of 3,072 features (32×32×3).

Pixel values were normalized to $[0, 1]$ and standardized using `StandardScaler`. Due to computational constraints, SVM and KNN were trained on a subset of 1,000 samples, while Logistic Regression used 4,000 samples. The validation set consisted of 1,000 samples.

Figure 8: Sample images from each of the 10 CIFAR-10 classes.

2.2 Logistic Regression

We evaluated regularization strength $C \in \{0.001, 0.01, 0.1, 1, 10\}$. The best validation accuracy of 38.6% was achieved at $C = 0.001$, indicating that strong regularization helps generalization on high-dimensional image data.

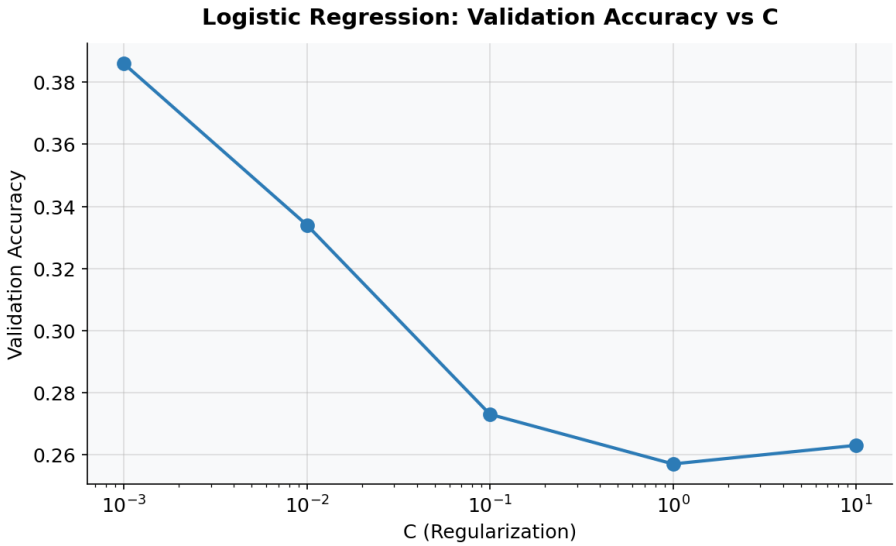

Figure 9: Logistic Regression: Validation Accuracy vs regularization parameter C.

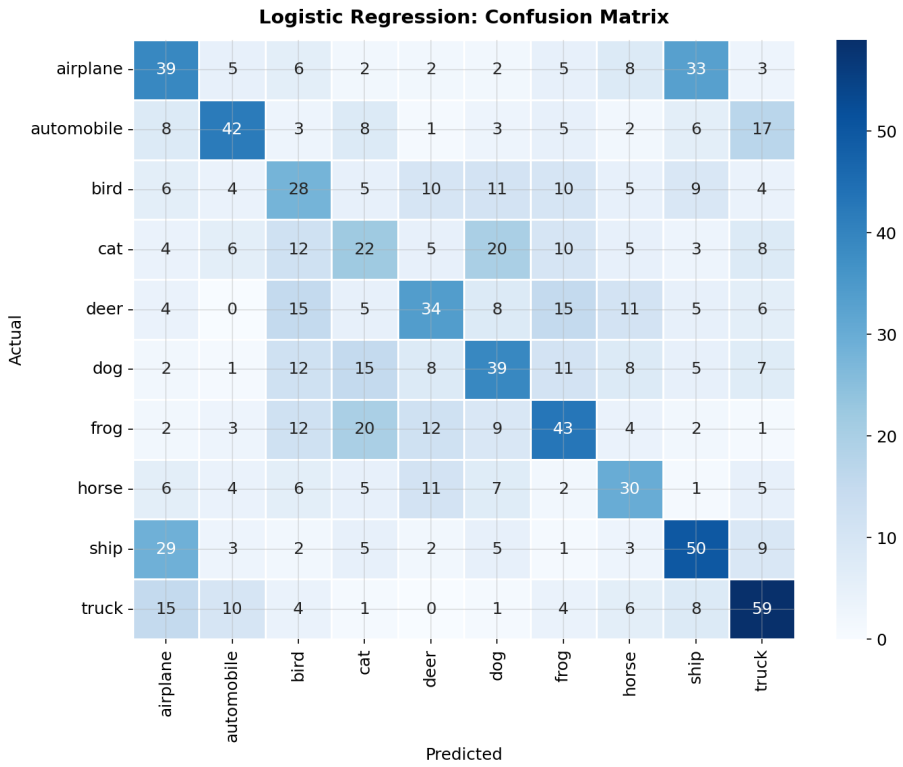

Figure 10: Logistic Regression: Confusion Matrix on validation set.

2.3 Support Vector Machine (SVM)

A Linear SVM was evaluated with the same range of C values. The best validation accuracy of 28.0% was achieved at $C = 0.001$. The lower performance compared to Logistic Regression is partly due to the smaller training subset (1,000 vs 4,000 samples).

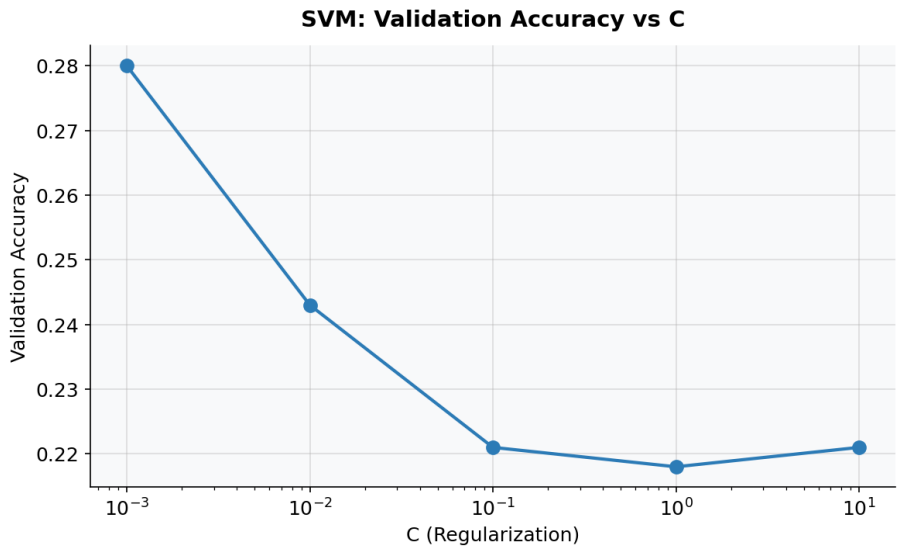

Figure 11: SVM: Validation Accuracy vs regularization parameter C.

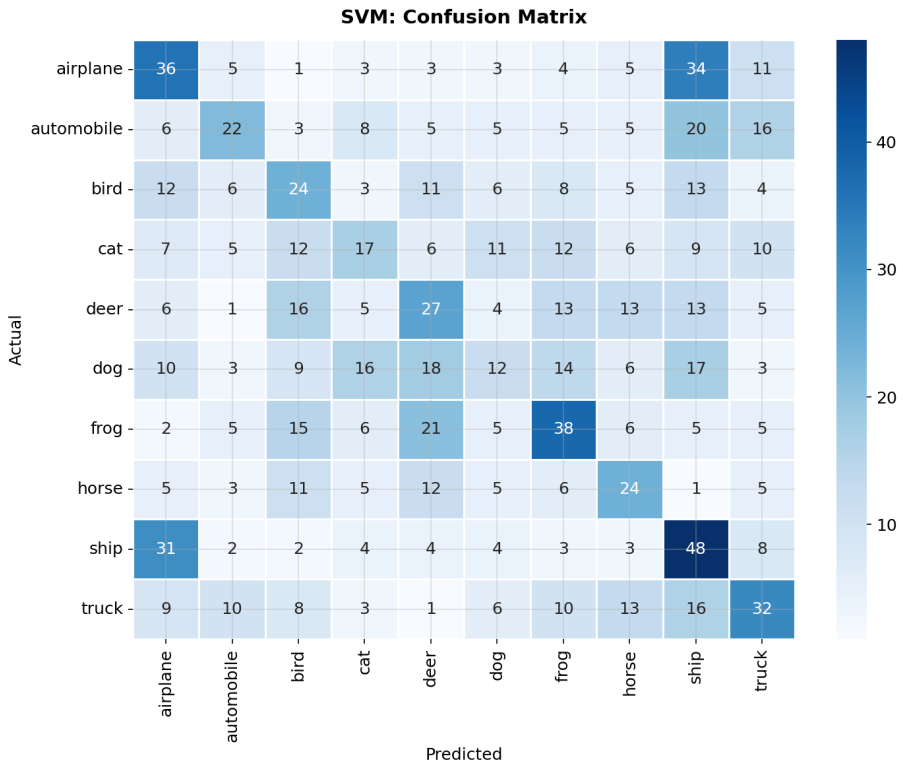

Figure 12: SVM: Confusion Matrix on validation set.

2.4 KNN Classification

KNN was evaluated for $k \in \{1, 3, 5, 7, 10\}$. The best validation accuracy of 26.5% was achieved at $k = 1$. KNN struggles with high-dimensional data due to the curse of dimensionality – in 3,072 dimensions, all points tend to be equally distant from each other, making nearest-neighbor search unreliable.

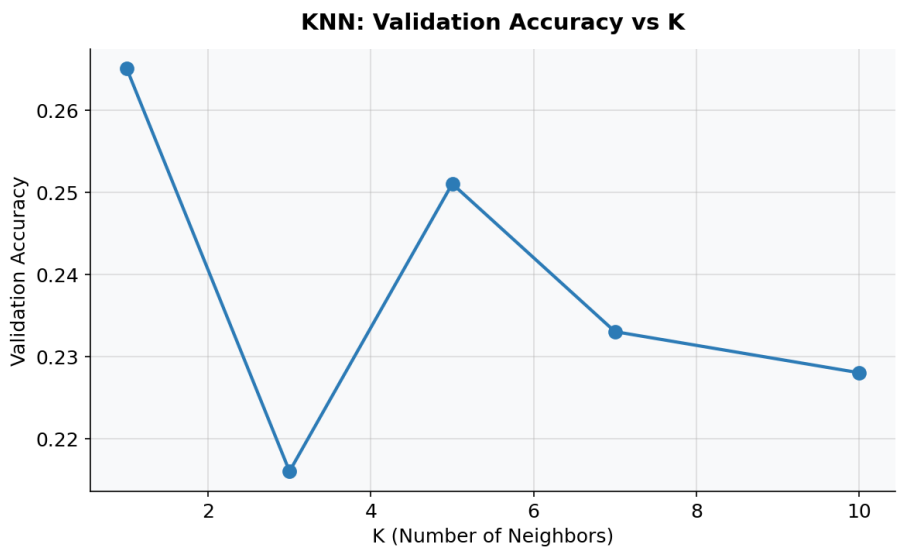

Figure 13: KNN: Validation Accuracy vs number of neighbors K.

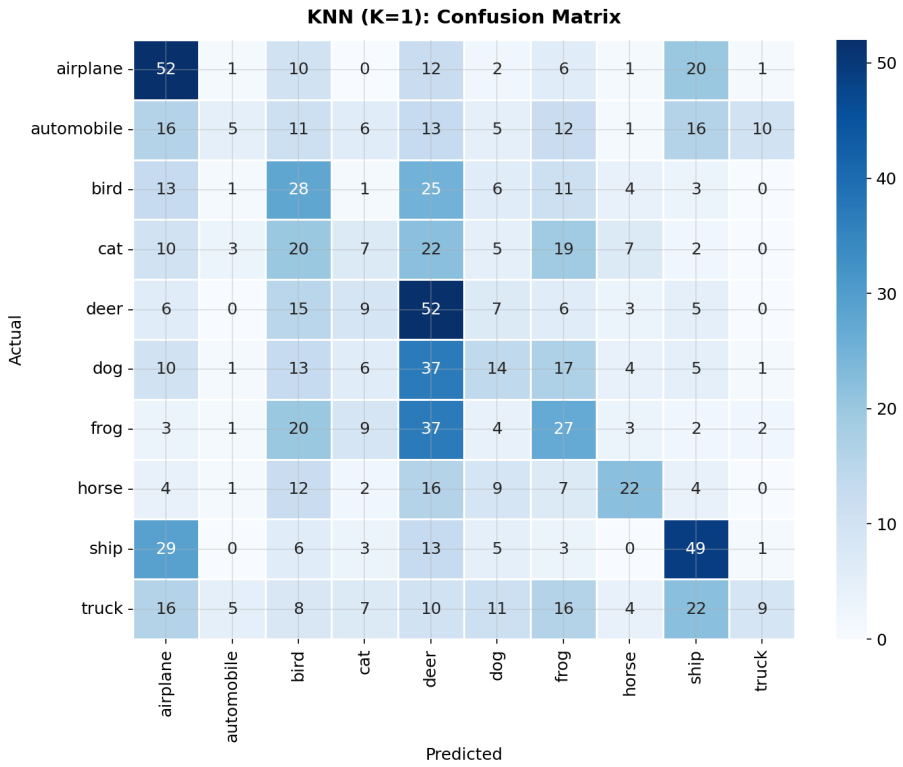

Figure 14: KNN (K=1): Confusion Matrix on validation set.

2.5 Model Comparison and Final Evaluation

Table 3 summarizes all models on the validation set.

Model	**Validation Accuracy**
Logistic Regression ($C = 0.001$)	0.386
SVM ($C = 0.001$)	0.280
KNN ($k = 1$)	0.265

Table 3: Model Comparison – Validation Set

Logistic Regression achieved the best validation accuracy and was selected as the final model. Evaluated on the held-out test set, it achieved an accuracy of 37.0%, confirming reasonable generalization.

All classical models perform significantly below state-of-the-art deep learning methods on CIFAR-10 (>90%), highlighting the limitations of linear models on high-dimensional image data.

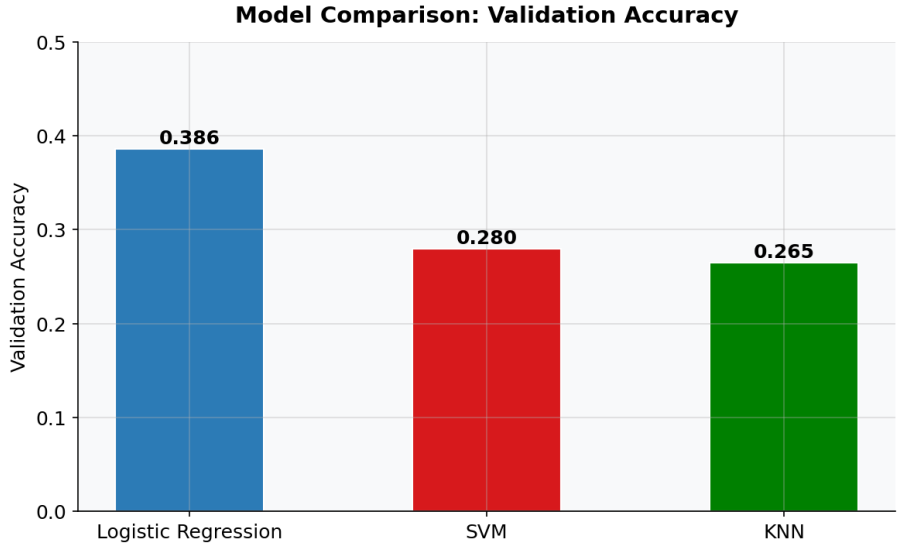

Figure 15: Model Comparison: Validation Accuracy across all three classifiers.

3 Neural Network with PyTorch

3.1 Overview

In this part, we implemented neural networks using PyTorch at three levels of abstraction on the CIFAR-10 dataset. All models were trained for one epoch (Parts II–IV) or ten epochs (Part V) using GPU acceleration (CUDA).

3.2 Barebones PyTorch (Part II)

We implemented a two-layer fully-connected network manually using raw PyTorch tensors, with Kaiming weight initialization and manual SGD updates. After one epoch, the model achieved approximately 45% validation accuracy, demonstrating that even a simple network can learn non-trivial structure in image data.

3.3 Module API (Part III)

We reimplemented the same two-layer network using `nn.Module`, which automatically tracks learnable parameters. This approach achieved similar accuracy (44%) with cleaner, more maintainable code.

3.4 Sequential API (Part IV)

Using `nn.Sequential`, we defined the same network more concisely and added Nesterov momentum to the SGD optimizer, achieving 45% validation accuracy.

3.5 Open-Ended Challenge – BetterNet (Part V)

To select the best architecture, we performed a hyperparameter search over 5 configurations, each trained for 3 epochs:

Config	Hidden	Layers	Dropout	Val Acc (3 epochs)
2-layer small	512	2	0.3	**0.532**
2-layer medium	1024	2	0.3	0.512
3-layer medium	1024	3	0.3	0.528
4-layer large	2048	4	0.3	0.528
4-layer low dropout	2048	4	0.2	0.529

Table 4: Hyperparameter Search – 3 Epoch Validation Accuracy

The 3-epoch search favors smaller networks that converge quickly. However, larger networks benefit more from extended training. We therefore selected the **4-layer large** architecture with dropout 0.2 (BetterNet) for full 10-epoch training, as it showed strong potential for improvement over time.

The final BetterNet architecture uses:

Layer	Size	Operations
FC 1	$3072 \rightarrow 2048$	BatchNorm, ReLU, Dropout(0.2)
FC 2	$2048 \rightarrow 1024$	BatchNorm, ReLU, Dropout(0.2)
FC 3	$1024 \rightarrow 512$	BatchNorm, ReLU, Dropout(0.2)
FC 4	$512 \rightarrow 10$	Output

Table 5: BetterNet Architecture

The model was trained using the Adam optimizer ($lr = 10^{-3}$, weight_decay $= 10^{-4}$) for 10 epochs. **Batch Normalization** stabilizes training by normalizing layer inputs. **Dropout** (rate 0.2) prevents overfitting by randomly deactivating neurons during training.

3.6 Results

Model	Val Accuracy	Test Accuracy
Barebones (Part II)	∼45%	–
Module API (Part III)	∼44%	–
Sequential (Part IV)	∼45%	–
BetterNet (Part V)	∼55%	**53.7%**

Table 6: PyTorch Models – Accuracy Comparison

BetterNet significantly outperforms the simpler models, demonstrating the benefit of depth, batch normalization, and dropout. The test accuracy of 53.7% also substantially outperforms the classical models from Part 2 (best: 38.6%), highlighting the advantage of neural networks for image classification tasks.

4 Bonus: Neural Network Regression with PyTorch

4.1 Overview

In this bonus task, we adapt the PyTorch classification notebook from Part 3 to solve the regression problem from Part 1 – predicting fuel efficiency (`mpg`) using the Auto MPG dataset. The key differences from the classification model are: (1) the output layer produces a single scalar value, (2) the loss function is MSE instead of cross-entropy, and (3) evaluation uses RMSE and R^2 instead of accuracy.

4.2 Architecture and Hyperparameter Search

We define a flexible **RegressionNet** with configurable hidden size, number of layers, and dropout rate. We compared 5 configurations using validation RMSE to select the best hyperparameters.

Hidden	**Layers**	**Dropout**	**LR**	**Val RMSE**
64	2	0.2	0.001	2.487
128	2	0.2	0.001	2.353
128	3	0.3	0.001	2.396
256	3	0.3	0.001	2.615
128	2	0.2	0.0005	**2.200**

Table 7: Hyperparameter Search Results – Validation RMSE

The best configuration uses hidden size 128, 2 layers, dropout rate 0.2, and learning rate 5×10^{-4}. Larger networks (256 units, 3 layers) performed worse, likely due to overfitting on the small dataset (278 training samples).

4.3 Final Model

The best model was trained for 100 epochs using the Adam optimizer ($lr = 5 \times 10^{-4}$, weight_decay $= 10^{-4}$) with Batch Normalization and Dropout(0.2). The architecture is:

- FC($7 \to 128$) $\to$ BatchNorm $\to$ ReLU $\to$ Dropout(0.2)
- FC($128 \to 128$) $\to$ BatchNorm $\to$ ReLU $\to$ Dropout(0.2)
- FC($128 \to 1$) – output layer

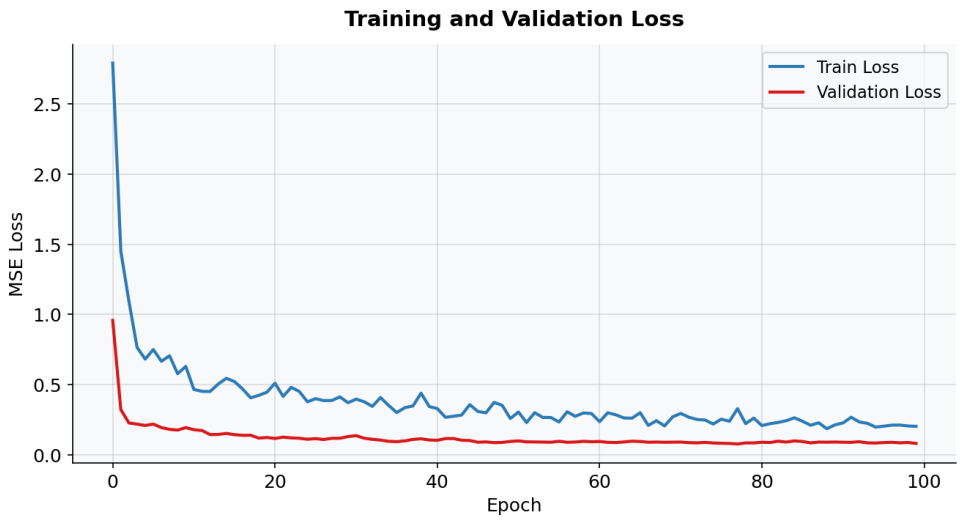

Figure 16: Training and Validation Loss over 100 epochs.

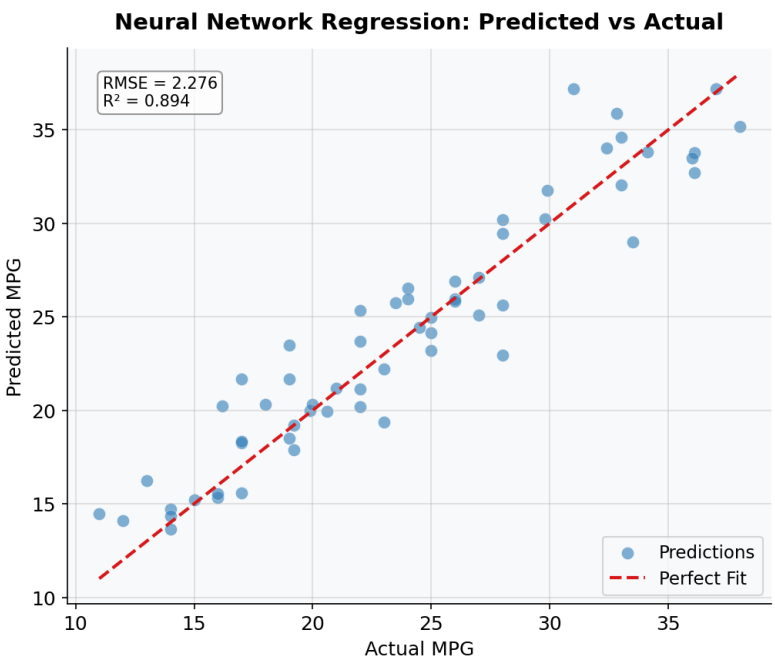

Figure 17: Neural Network Regression: Predicted vs Actual MPG on validation set.

4.4 Comparison with Part 1

Table 8 compares the neural network regression model with the best model from Part 1 (KNN, $k = 4$).

Model	**Test RMSE**	**Test R^2**
KNN ($k = 4$) – Part 1	2.400	0.912
Neural Network – Bonus	**2.340**	**0.905**

Table 8: Comparison: Neural Network vs KNN Regression

The neural network achieves a lower test RMSE (2.340 vs 2.400), outperforming the best classical model from Part 1. This demonstrates that even on small tabular datasets, a well-regularized neural network can match or exceed classical methods. The loss curve shows smooth convergence without overfitting, confirming that Batch Normalization and Dropout were effective regularization strategies.

Conclusion

This project explored supervised learning across regression and classification tasks. Key findings include:

- **Part 1:** KNN ($k = 4$) achieved the best regression performance (Test RMSE = 2.400, $R^2 = 0.912$), outperforming linear and polynomial models.
- **Part 2:** Logistic Regression achieved the best classification accuracy (37.0% on test set), highlighting the limitations of classical models on high-dimensional image data.
- **Part 3:** The custom BetterNet architecture with Batch Normalization and Dropout achieved 53.7% test accuracy, significantly outperforming classical classifiers.
- **Bonus:** A neural network regression model achieved Test RMSE = 2.340, slightly outperforming the best classical model from Part 1.

Overall, neural networks consistently outperformed classical models, particularly on image data, while classical models remained competitive on small tabular datasets.